



Delft University of Technology

KapCie

Tymon Cichocki, Wiktor Cupiał, Zofia Krok

2025-09-08

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

template.cpp

Description: template

<bits/stdc++.h>89dca5, 46 lines

using namespace std;
#define mp make_pair
#define pb push_back
#define st first
#define nd second
#define fi first
#define se second
#define vt vector
#define FOR(a, b, c) for(int a=b; a<c; ++a)
#define all(a) (a).begin(), (a).end()
#define sz(a) (int) (a).size()

typedef long long ll;
typedef long double ld;
typedef pair<ll, ll> pll;
typedef pair<int, int> pii;
typedef pii PII;
typedef pll PLL;
typedef vector<int> vi;
typedef vector<ll> vll;

constexpr ll nax = 2e5+6969, INF = 1e9+2137;
constexpr ld eps = 1e-9;

#ifndef DEBUG
#define DTP(x, y) auto operator << (auto &o, auto a) -> decltype(y, o) { o << "{"; x; return o << "}; }
DTP(o << a.st << ", " << a.nd, a.nd);
DTP(int j=0; for(auto i : a) o << i << ((++j) != sz(a) ? ", " : " "), all(a));
void dump(auto... x) { ((cerr << x << ", "), ...) << '\n'; }
#define debug(x...) cerr << setw(4) << __LINE__ << ": [" #x "]: "
, dump(x)
#else
#define debug(...) 0
#endif

1mtl19937_64 rng(6969);
1// mtl19937_64 rng(chrono::system_clock::now().time_since_epoch().count());
3inline ll rand(ll l, ll r) {
3return uniform_int_distribution<ll>(l, r)(rng);
}
6int32_t main() {
9ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
11return 0;
}
12flags.txt4 lines
18db() {
18g++ -Wall -Wextra -Wshadow -Wconversion -pedantic -std=c++17 -g -Og -fno-sanitize-recover -fstack-protector -fsanitize=address,undefined -DDEBUG "\$1".cpp -o "\$@"
22}
22// -D_GLIBCXX_DEBUG
24.vimrc6 lines
set cin ai is ts=4 sw=4 nu mouse=a
sy on
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \l tr -d '[:space:]' \ \l md5sum \l cut -c-6
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Mathematics (2)

2.1 Equations

$$ax^2+bx+c=0\Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

The extremum is given by $x=-b/2a$.

$$\begin{matrix} ax+by=e & x=\frac{ed-bf}{ad-bc} \\ cx+dy=f & \Rightarrow y=\frac{af-ec}{ad-bc} \end{matrix}$$

In general, given an equation $Ax=b$, the solution to a variable x_i is given by

$$x_i=\frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n=c_1a_{n-1}+\cdots+c_ka_{n-k}$, and $r_1,\dots,r_k$ are distinct roots of $x^k+c_1x^{k-1}+\cdots+c_k$, there are $d_1,\dots,d_k$ s.t.

$$a_n=d_1r_1^n+\cdots+d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g.
 $a_n=(d_1n+d_2)r^n$.

2.3 Trigonometry

$\sin(v + w) = \sin v \cos w + \cos v \sin w$
 $\cos(v + w) = \cos v \cos w - \sin v \sin w$

$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$
 $\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$
 $\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$

$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$
where V, W are lengths of sides opposite angles v, w .

$a \cos x + b \sin x = r \cos(x - \phi)$
 $a \sin x + b \cos x = r \sin(x + \phi)$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - \frac{2bc \cos \alpha}{\frac{\alpha + \beta}{2}}$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

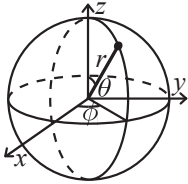
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$

For cyclic quadrilaterals the sum of opposite angles is 180° ,
 $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.4.3 Spherical coordinates



$x = r \sin \theta \cos \phi$
 $y = r \sin \theta \sin \phi$
 $z = r \cos \theta$
 $r = \sqrt{x^2 + y^2 + z^2}$
 $\theta = \text{acos}(z / \sqrt{x^2 + y^2 + z^2})$
 $\phi = \text{atan2}(y, x)$

2.5 Derivatives/Integrals

$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1 - x^2}}$
 $\frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1 - x^2}}$
 $\frac{d}{dx} \tan x = 1 + \tan^2 x$
 $\frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$
 $\int \tan ax = -\frac{\ln |\cos ax|}{a}$
 $\int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$
 $\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \text{erf}(x)$
 $\int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax - 1)$

Integration by parts:

$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$

2.6 Sums

$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$

$1 + 2 + 3 + \dots + n = \frac{n(n + 1)}{2}$
 $1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$
 $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n + 1)^2}{4}$
 $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$

2.7 Series

$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$
 $\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$
 $\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$

$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$
 $\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$

For independent X and Y ,

$V(aX + bY) = a^2V(X) + b^2V(Y)$.

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots, 0 \leq p \leq 1$.

$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$

$\mu = np, \sigma^2 = np(1 - p)$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each wich yields success with probability p is $\text{Fs}(p)$, $0 \leq p \leq 1$.

$p(k) = p(1 - p)^{k - 1}, k = 1, 2, \dots$

$\mu = \frac{1}{p}, \sigma^2 = \frac{1 - p}{p^2}$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$

$\mu = \lambda, \sigma^2 = \lambda$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $U(a,b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

MapComparator.h

Description: example of function object (functor); keeps default behavior cmp(a, a) should always return FALSE(!)
Usage: set<int,cmp> s; map<int,int,cmp> m;

065243, 1 lines

struct cmp{bool operator() (int l,int r){return l<r;}};

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_t.type.
Time: $\mathcal{O}(\log N)$

782797, 16 lines

#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

void example() {
Tree<int> t, t2; t.insert(8);
auto it = t.insert(10).first;
assert(it == t.lower_bound(9));
assert(t.order_of_key(10) == 1);
assert(t.order_of_key(11) == 2);
assert(*t.find_by_order(0) == 8);
t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}

HashMap.h

Description: Hash map with the same API as unordered_map, but ~3x faster. Initial capacity must be a power of 2 if provided.
Usage: ht<int,int> h{{},{},{},{},{1<<16}};

<ext/pb_ds/assoc.container.hpp>

a37e68, 10 lines

using namespace __gnu_pbds;
struct chash {
const uint64_t C = 1l(2e18*PI)+71; // large odd number
const int RANDOM = rng();
ll operator()(ll x) const {
return __builtin_bswap64((x^RANDOM)*C); }
};
template<class K,class V> using ht = gp_hash_table<K,V,chash>;
template<class K,class V> V get(ht<K,V>& u, K x) {
auto it = u.find(x); return it == end(u) ? 0 : it->s; }
}

SegmentTree.h

Description: Zero-indexed segtree. Queries [,).

Usage: segtree<int, op, e> seg(vec);

a5c856, 25 lines

template <class S, S (*op)(S, S), S (*e)()> struct segtree {
private:
int n, size, log;
vector<S> d;
void update(int k) {d[k] = op(d[2 * k], d[2 * k + 1]);}
public:
segtree(const vector <S>&v) : n(sz(v)) {

```
log = (32 - __builtin_clz(n)); size = 1 << log;  
d = vector<S>(2*size, e());  
rep(i, 0, n) d[size+i] = v[i];  
for (int i = size - 1; i >= 1; --i) update(i);  
}  
void set(int pos, S val) {  
pos += size; d[pos] = val;  
rep(i, 1, log + 1) update(pos >> i);  
}  
S prod(int l, int r) {  
S sml = e(), smr = e();  
for (l += size, r += size; l < r; l >= 1, r >= 1) {  
if (l & 1) sml = op(sml, d[l++]);  
if (r & 1) smr = op(d[--r], smr);  
}  
return op(sml, smr);  
}  
};
```

LazySegmentTree.h

Description: Segment tree with good lazy propagation templates. Queries [,). You give two monoids and one acts on the other by monoid homomorphisms.

Usage: Couting bitwise inversions on ranges:

```
struct S {mint a; int size;};  
struct F {mint a, b;};  
S op(S l, S r) { return S{l.a + r.a, l.size + r.size}; }  
S e() { return S{0, 0}; }  
S mapping(F l, S r) { return S{r.a * l.a + r.size * l.b,  
r.size}; }  
F composition(F l, F r) { return F{r.a * l.a, r.b * l.a + l.b};  
}  
F id() { return F{1, 0}; }
```

lazy_segtree<S, op, e, F, mapping, composition, id> seg(vec);
Time: $\mathcal{O}(\log N)$.

ab8443, 62 lines

```
template <class S, S(*op)(S, S), S(*e)(),  
class F, S (*mapping)(F, S), F(*composition)(F, F), F (*id)()>  
struct lazy_segtree {  
private:  
int n, size, log;  
vector <S> d; vector <F> lz;  
void update(int k) {d[k] = op(d[k << 1], d[k << 1 | 1]); }  
void all_apply(int k, F f) {  
d[k] = mapping(f, d[k]);  
if (k < size) lz[k] = composition(f, lz[k]);  
}  
void push(int k) {  
all_apply(k << 1, lz[k]);  
all_apply(k << 1 | 1, lz[k]);  
lz[k] = id();  
}  
void make_pushes(int &l, int &r) {  
l += size; r += size;  
for (int i = log; i > 0; --i) {  
if (((l >> i) << i) != l) push(l >> i);  
if (((r >> i) << i) != r) push(r >> i);  
}  
}  
public:  
lazy_segtree(const vector <S> &v) : n(sz(v)) {  
log = 32 - __builtin_clz(n); size = 1 << log;  
d = vector<S>(2*size, e());  
lz = vector<F>(size, id());  
rep(i, 0, n) d[size + i] = v[i];  
for (int i=size-1; i>0; --i) update(i);  
}  
void set(int p, S x) {
```

```

    p += size;
    for (int i = log; i >= 1; i--) push(p >> i);
    d[p] = x;
    for (int i = 1; i <= log; i++) update(p >> i);
}
S prod(int l, int r) {
    if (l >= r) return e();
    make_pushes(l, r);
    S sml = e(), smr = e();
    for (; l < r; l >>= 1, r >>= 1) {
        if (l & 1) sml = op(sml, d[l++]);
        if (r & 1) smr = op(d[--r], smr);
    }
    return op(sml, smr);
}
void apply(int l, int r, F f) {
    if (l >= r) return;
    make_pushes(l, r);
    int initl = l, initr = r;
    for (; l < r; l >>= 1, r >>= 1) {
        if (l & 1) all_apply(l++, f);
        if (r & 1) all_apply(--r, f);
    }
    l = initl; r = initr;
    rep(i, l, log+1) {
        if ((l >> i) << i) != 1) update(l >> i);
        if ((r >> i) << i) != r) update((r - 1) >> i);
    }
}
};

```

PersistentSegtree.h

Description: segment tree with historical data

Usage: figure it out

Time: $\mathcal{O}((\log N))$ per operation

c4782a, 37 lines

```

struct Vertex {
    Vertex *l, *r;
    int sum;

    Vertex(int val) : l(nullptr), r(nullptr), sum(val) {}
    Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {
        if (l) sum += l->sum;
        if (r) sum += r->sum;
    }
};

Vertex* build(int a[], int tl, int tr) {
    if (tl == tr)
        return new Vertex(a[tl]);
    int tm = (tl + tr) / 2;
    return new Vertex(build(a, tl, tm), build(a, tm+1, tr));
}

int get_sum(Vertex* v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && tr == r)
        return v->sum;
    int tm = (tl + tr) / 2;
    return get_sum(v->l, tl, tm, l, min(r, tm))
        + get_sum(v->r, tm+1, tr, max(l, tm+1), r);
}

Vertex* update(Vertex* v, int tl, int tr, int pos, int new_val)
{
    if (tl == tr)
        return new Vertex(new_val);
    int tm = (tl + tr) / 2;

```

```

    if (pos <= tm)
        return new Vertex(update(v->l, tl, tm, pos, new_val), v->r);
    else
        return new Vertex(v->l, update(v->r, tm+1, tr, pos, new_val));
}

```

UnionFind.h

Description: Disjoint-set data structure.

Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```

struct UF {
    vi e;
    UF(int n) : e(n, -1) {}
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        e[a] += e[b]; e[b] = a;
        return true;
    }
};

```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().

Usage: int t = uf.time(); ...; uf.rollback(t);

Time: $\mathcal{O}(\log N))$

de4ad0, 21 lines

```

struct RollbackUF {
    vi e; vector<pii> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i --> t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};

```

Matrix.h

Description: Basic operations on square matrices.

Usage: Matrix<int, 3> A;

A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};

vector<int> vec = {1,2,3};

vec = (A^N) * vec;

c43c7d, 26 lines

```

template<class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i,0,N) rep(j,0,N)
            rep(k,0,N) a.d[i][j] += d[i][k]*m.d[k][j];
    }
};

```

```

    return a;
}
vector<T> operator*(const vector<T>& vec) const {
    vector<T> ret(N);
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
}
M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
        if (p&1) a = a*b;
        b = b*b;
        p >>= 1;
    }
    return a;
}
};

```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).

Time: $\mathcal{O}(\log N)$

Sec1c7, 30 lines

```

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

```

Lichao.h

Description: LiChao tree for dynamic convex hull trick. can query at any point and insert lines/segments on ranges.

Time: $\mathcal{O}(n \log n)$

f3f19f, 66 lines

```

template <typename T>
struct LichaoTree {
    const T INF = std::numeric_limits<T>::max();
    struct line {
        T a, b;
        line(T a, T b) : a(a), b(b) {}
        T operator()(T x) const { return a * x + b; }
    };
};

```

```
};
int n;
std::vector<line> fs;
std::vector<T> xs;

int index(T x) const { return lower_bound(xs.begin(), xs.end()
(), x) - xs.begin(); }

void update(T a, T b, int l, int r) {
    line g(a, b);
    for(l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if(l & 1) descend(g, l++);
        if(r & 1) descend(g, --r);
    }
}

void descend(line g, int i) {
    int l = i, r = i + 1;
    while(l < n) l <= 1, r <= 1;
    while(l < r) {
        int c = (l + r) >> 1;
        T xl = xs[l - n], xr = xs[r - 1 - n], xc = xs[c - n];
        line &f = fs[i];
        if(f(xl) <= g(xl) && f(xr) <= g(xr)) return;
        if(f(xl) >= g(xl) && f(xr) >= g(xr)) {
            f = g;
            return;
        }
        if(f(xc) > g(xc)) swap(f, g);
        if(f(xl) > g(xl))
            i = i << 1 | 0, r = c;
        else
            i = i << 1 | 1, l = c;
    }
}

public:
Tree(const std::vector<T> &xs_) : xs(xs_) {
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
    n = xs.size();
    fs.assign(n << 1, line(T(0), INF));
}

// add f(x) = ax + b
void add_line(T a, T b) { update(a, b, 0, n); }

// add f(x) = ax + b (x in [xl, xr])
void add_segment(T a, T b, T xl, T xr) {
    int l = index(xl), r = index(xr);
    update(a, b, l, r);
}

T get_min(T x) const {
    int i = index(x);
    T res = INF;
    for(i += n; i; i >>= 1) res = min(res, fs[i](x));
    return res;
}
};
```

Treap.h
Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
Time: $\mathcal{O}(\log N)$

```
struct Node {
    Node *c[2] = {0, 0};
```

```
int prio, s = 1; // by default this just counts number of
                children
Node() : prio(rand()) {}
void recalc();
};

int sum(Node* n) { return n ? n->s: 0; }
void Node::recalc() { // push lazy propagation here
    s = sum(c[0]) + 1 + sum(c[1]);
}

Node* attach(Node *l, Node* n, Node *r){
    n->c[0] = l, n->c[1] = r;
    n->recalc();
    return n;
}

pair<Node*, Node*> split(Node* n, int k) {
    if (!n) return {};
    n->recalc();
    if (sum(n->c[0]) >= k) { // "n->val >= k" for lower_bound(k)
        auto [l, r] = split(n->c[0], k);
        return {l, attach(r, n, n->c[1])};
    } else {
        auto [l, r] = split(n->c[1], k - 1 - sum(n->c[0])); // and
        just "k"
        return {attach(n->c[0], n, l), r};
    }
}

Node* merge(Node* l, Node* r) {
    if (!l) return r;
    if (!r) return l;
    l->recalc(); // only needed for lazy propagation
    r->recalc();
    return l->prio > r->prio ?
        attach(l->c[0], l, merge(l->c[1], r)) :
        attach(merge(l, r->c[0]), r, r->c[1]);
}
```

FenwickTree2d.h
Description: Computes sums $a[i,j]$ for all $i < I, j < J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

```
"FenwickTree.h" 157f07, 22 lines

struct FT2 {
    vector<vi> ys; vector<FT> ft;
    FT2(int limx) : ys(limx) {}
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
    }
    void init() {
        for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x-1].query(ind(x-1, y));
        return sum;
    }
};
```

BIT.h
Description: range sum queries and point updates for D dimensions
Usage: {BIT<int,10,10>} gives 2D BIT
Time: $\mathcal{O}\left((\log N)^D\right)$

```
1cb741, 14 lines

template <class T, int ...Ns> struct BIT {
    T val = 0; void upd(T v) { val += v; }
    T query() { return val; }
};

template <class T, int N, int... Ns> struct BIT<T, N, Ns...> {
    BIT<T,Ns...> bit[N+1];
    template<typename... Args> void upd(int pos, Args... args) {
        for (; pos<=N; pos+=pos&-pos) bit[pos].upd(args...); }
    template<typename... Args> T sum(int r, Args... args) {
        T res=0; for (;r;r-=r&-r) res += bit[r].query(args...);
        return res; }
    template<typename... Args> T query(int l, int r, Args...
        args) { return sum(r,args...)-sum(l-1,args...); }
};
```

RMQ.h
Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a + 1], \dots V[b - 1])$ in constant time.
Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V| \log |V| + Q)$

```
510c32, 16 lines

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

MoQueries.h
Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a,c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

```
a12ef4, 49 lines

void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }
}
```

```
    return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) rep(end,0,2) {
        int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
        while (!(L[b] <= L[a] && R[a] <= R[b]))
            I[i++] = b, b = par[b];
        while (a != b) step(par[a]);
        while (i--) step(I[i]);
        if (end) res[qi] = calc();
    }
    return res;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.

Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0

Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h"

b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin-1);
```

```
dr.push_back(xmax+1);
sort(all(dr));
rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
        rep(it,0,60) { // while (h - l > 1e-8)
            double m = (l + h) / 2, f = p(m);
            if ((f <= 0) ^ sign) l = m;
            else h = m;
        }
        ret.push_back((l + h) / 2);
    }
}
return ret;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.

Time: $\mathcal{O}(n^2)$

08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first 2n terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.

Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

Time: $\mathcal{O}(N^2)$

"../number-theory/ModPow.h"

96548b, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.

Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number

Time: $\mathcal{O}(n^2 \log k)$

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval [a,b] assuming f is unimodal on the interval, i.e. has only one local minimum. The maximum error in the result is eps. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.

Usage: double func(double x) { return 4*x+.3*x*x; }

double xmin = gss(-1000,1000,func);

Time: $\mathcal{O}(\log((b-a)/\epsilon))$

31d45b, 14 lines

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions

Seeeaf, 14 lines

typedef array<double, 2> P;

```
template<class F> pair<double, P> hillClimb(P start, F f) {
    pair<double, P> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
```

```
    P p = cur.second;
    p[0] += dx*jmp;
    p[1] += dy*jmp;
    cur = min(cur, make_pair(f(p), p));
}
}
return cur;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

4756fc, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    rep(i,1,n*2)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage: double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});};});};

92dd79, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}

template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b, x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vvd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: $\mathcal{O}(NM * \text{\#pivots})$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

aa8530, 68 lines

```
typedef double T; // long double, Rational, double + modKP>...
typedef vector<T> vd;
typedef vector<vd> vvd;
```

```
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
```

```
struct LPSolver {
    int m, n;
```

```
    vi N, B;
    vvd D;

    LPSolver(const vvd& A, const vd& b, const vd& c) :
        m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
        rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m+1][n] = 1;
    }
```

```
    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
            T *b = D[i].data(), inv2 = b[s] * inv;
            rep(j,0,n+2) b[j] -= a[j] * inv2;
            b[s] = a[s] * inv2;
        }
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
```

```
    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
            if (D[x][s] >= -eps) return true;
            int r = -1;
            rep(i,0,m) {
                if (D[i][s] <= eps) continue;
                if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                    < MP(D[r][n+1] / D[r][s], B[r])) r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }
```

```
T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
        pivot(r, n);
        if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
        rep(i,0,m) if (B[i] == -1) {
            int s = 0;
            rep(j,1,n+1) ltj(D[i]);
            pivot(i, s);
        }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
}
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
```

```
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.

Time: $\mathcal{O}(N^3)$

3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

44c9ab, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vi col(m); iota(all(col), 0);

    rep(i,0,n) {
        double v, bv = 0;
        rep(r,i,n) rep(c,i,m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) swap(A[j][i], A[j][bc]);
        bv = 1/A[i][i];
        rep(j,i+1,n) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            rep(k,i+1,m) A[j][k] -= fac*A[i][k];
        }
        rank++;
    }
}
```



```
x.assign(m, 0);
for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
}
return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h"	08e495, 7 lines
-----------------	-----------------

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .

Time: $\mathcal{O}(n^2m)$

typedef bitset<1000> bs;	fa2d7a, 34 lines
--------------------------	------------------

```
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        for (br=1; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if(b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
            A[j] ^= A[i];
        }
        rank++;
    }
}
```

```
x = bs();
for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
}
return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod p$, and k is doubled in each step.

Time: $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
int matInv(vector<vector<double>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;

    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j,i+1,n) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] -= f*A[i][k];
            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j,i+1,n) A[i][j] /= v;
        rep(j,0,n) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for (int i = n-1; i > 0; --i) rep(j,0,i) {
        double v = A[j][i];
        rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
    }
}
```

```
rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d,p,q,b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, \, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for $\text{diag}[i] == 0$ is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        }
    }
}
```

```
    } else {
        diag[i+1] -= super[i]*sub[i]/diag[i];
        b[i+1] -= b[i]*sub[i]/diag[i];
    }
}
for (int i = n; i--;) {
    if (tr[i]) {
        swap(b[i], b[i-1]);
        diag[i-1] = diag[i];
        b[i] /= super[i-1];
    } else {
        b[i] /= diag[i];
        if (i) b[i-1] -= b[i]*super[i-1];
    }
}
return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , $\text{reverse}(\text{start}+1, \text{end})$, FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h"b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod].

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h"ced03d, 33 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
}
```

```
    return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

464cf3, 16 lines

```
void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
        if (inv) for (int& x : a) x /= sz(a); // XOR only
    }
    vi conv(vi a, vi b) {
        FST(a, 0); FST(b, 0);
        rep(i,0,sz(a)) a[i] *= b[i];
        FST(a, 1); return a;
    }
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h"690185, 34 lines

```
template<ll mod>
struct mint {
    ll x;
    mint() : x(0) {}
    mint(ll x) : x(x) {}

    mint& operator+=(mint const& v) { x += v.x; if(x >= mod) x -= mod; return *this; }
    mint& operator-=(mint const& v) { x -= v.x; if(x < 0) x += mod; return *this; }
    mint& operator*=(mint const& v) { x = (x*v.x)%mod; return *this; }
    mint& operator/=(mint const& v) { return *this *= v.inv(); }
    friend mint operator+(mint const& l, mint const& r) { return mint(l) += r; }
    friend mint operator-(mint const& l, mint const& r) { return mint(l) -= r; }
    friend mint operator*(mint const& l, mint const& r) { return mint(l) *= r; }
    friend mint operator/(mint const& l, mint const& r) { return mint(l) /= r; }
    friend bool operator<(mint const& l, mint const& r) { return l.x < r.x; }
    friend bool operator==(mint const& l, mint const& r) { return l.x == r.x; }
    friend bool operator!=(mint const& l, mint const& r) { return l.x != r.x; }
    mint inv() const { return bpow(x, mod-2); }
}
```

```
static mint bpow(mint a, ll b) {
    mint res = 1;
    while(b > 0) {
        if(b&1)
```

```
        res = res*a;
        a = a*a;
        b >>= 1;
    }
    return res;
}

friend ostream& operator<<(ostream& out, const mint& v) {
    return out << "mint{" << v.x << ", " << mod << "}";
}
};
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM ≤ mod and that mod is a prime.

6f684f, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

modsum(to, c, k, m) = $\sum_{i=0}^{to-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: log(m), with a large constant.

5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \pmod c$ (or $a^b \pmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

bbbd8f, 11 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
```

```
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ (− x gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h"	19a793, 24 lines
<pre>ll sqrt(ll a, ll p) { a %= p; if (a < 0) a += p; if (a == 0) return 0; assert(modpow(a, (p-1)/2, p) == 1); // else no solution if (p % 4 == 3) return modpow(a, (p+1)/4, p); // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5 ll s = p - 1, n = 2; int r = 0, m; while (s % 2 == 0) ++r, s /= 2; while (modpow(n, (p - 1) / 2, p) != p - 1) ++n; ll x = modpow(a, (s + 1) / 2, p); ll b = modpow(a, s, p), g = modpow(n, s, p); for (; r = m) { ll t = b; for (m = 0; m < r && t != 1; ++m) t = t * t % p; if (m == 0) return x; ll gs = modpow(g, 1LL << (r - m - 1), p); g = gs * gs % p; x = x * gs % p; b = b * g % p; } }</pre>	

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

	6b2912, 20 lines
<pre>const int LIM = 1e6; bitset<LIM> isPrime; vi eratosthenes() { const int S = (int)round(sqrt(LIM)), R = LIM / 2; vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1)); vector<pii> cp; for (int i = 3; i <= S; i += 2) if (!sieve[i]) { cp.push_back({i, i * i / 2}); for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1; } for (int L = 1; L <= R; L += S) { array<bool, S> block{}; for (auto &[p, idx] : cp) for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1; rep(i,0,min(S, R - L)) if (!block[i]) pr.push_back((L + i) * 2 + 1); } for (int i : pr) isPrime[i] = 1; return pr; }</pre>	

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \pmod c$.

"ModMulLL.h"	60dcd1, 12 lines
<pre>bool isPrime(ull n) { if (n < 2 n % 6 % 4 != 1) return (n 1) == 3; ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022}, s = __builtin_ctzll(n-1), d = n >> s; for (ull a : A) { // ^ count trailing zeroes ull p = modpow(a%n, d, n), i = s; while (p != 1 && p != n - 1 && a % n && i--) p = modmul(p, p, n); if (p != n-1 && i != s) return 0; } return 1; }</pre>	

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h"	a33cf6, 18 lines
<pre>ull pollard(ull n) { auto f = [n](ull x) { return modmul(x, x, n) + 1; }; ull x = 0, y = 0, t = 30, prd = 2, i = 1, q; while (t++ % 40 __gcd(prd, n) == 1) { if (x == y) x = ++i, y = f(x); if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q; x = f(x), y = f(f(y)); } return __gcd(prd, n); } vector<ull> factor(ull n) { if (n == 1) return {}; if (isPrime(n)) return {n}; ull x = pollard(n); auto l = factor(x), r = factor(n / x); l.insert(l.end(), all(r)); return l; }</pre>	

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

	33ba8f, 5 lines
<pre>ll euclid(ll a, ll b, ll &x, ll &y) { if (!b) return x = 1, y = 0, a; ll d = euclid(b, a % b, y, x); return y -= a/b * x, d; }</pre>	

Euclid2.h

Description: finds smallest $x \geq 0$ such that $L \leq Ax \pmod P \leq R$

	063047, 9 lines
--	-----------------

```
ll cdiv(ll x, ll y) { return (x+y-1)/y; }
ll bet(ll P, ll A, ll L, ll R) {
    if (A == 0) return L == 0 ? 0 : -1;
    ll c = cdiv(L,A); if (A*c <= R) return c;
    ll B = P%A; // P = k*A+B, L <= A(x-Ky)-By <= R
    // => -R <= By % A <= -L
    auto y = bet(A,B,A-R%A,A-L%A);
    return y == -1 ? y : cdiv(L+B*y,A)+P/A*y;
}
```

CRT.h

Description: Chinese Remainder Theorem.

`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

"euclid.h"	04d93a, 7 lines
<pre>ll crt(ll a, ll m, ll b, ll n) { if (n > m) swap(a, b), swap(m, n); ll x, y, g = euclid(m, n, x, y); assert((a - b) % g == 0); // else no solution x = (b - a) % n * x % n / g * m + a; return x < 0 ? x + m*n/g : x; }</pre>	

5.3.1 Bézout’s identity

For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler’s ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p \text{ prime} \Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$

Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.

Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

	cf7d6d, 8 lines
<pre>const int LIM = 5000000; int phi[LIM];</pre>	

<pre>void calculatePhi() { rep(i,0,LIM) phi[i] = i&1 ? i : i/2; for (int i = 3; i < LIM; i += 2) if(phi[i] == i) for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i; }</pre>	
---	--

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.

Time: $\mathcal{O}(\log N)$

	dd6c5e, 21 lines
<pre>typedef double d; // for N ~ 1e7; long double for N ~ 1e9 pair<ll, ll> approximate(d x, ll N) { ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x; for (;) { ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf), a = (ll)floor(y), b = min(a, lim), NP = b*P + LP, NQ = b*Q + LQ; if (a > b) { // If b > a/2, we have a semi-convergent that gives us a // better approximation; if b = a/2, we *may* have one. // Return {P, Q} here for a more canonical approximation. return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ? make_pair(NP, NQ) : make_pair(P, Q); } } }</pre>	

```

    if (abs(y = 1/(y - (d)a)) > 3*N) {
        return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
}
}
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0,1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `fracBS([])(Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`
Time: $\mathcal{O}(\log(N))$

27ab3e, 25 lines

```

struct Frac { ll p, q; };
```

```

template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert (f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !!adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d \mid n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d \mid n} f(d) \Leftrightarrow f(n) = \sum_{d \mid n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$$\sum_{d \mid n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n \mid d} f(d) \Leftrightarrow f(n) = \sum_{n \mid d} \mu(d/n) g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.

Time: $\mathcal{O}(n)$

044568, 6 lines

```

int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for (int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k \mid n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2e5$	$\sim 2e8$

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$.

a0a312, 6 lines

```

ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i, 1, sz(v)) rep(j, 0, v[i])
        c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$$B[0, \dots] = [1, -\tfrac{1}{2}, \tfrac{1}{6}, 0, -\tfrac{1}{30}, 0, \tfrac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x) dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles. Alternatively, number of permutations on n items with k prefix maxima. You can compute $c(n, k)$ in $O(n\log^2)$ using the second formula, D& C and FFT.

$$c(n, k) = c(n - 1, k - 1) + (n - 1)c(n - 1, k), \quad c(0, 0) = 1$$
$$\sum_{k=0}^n c(n, k)x^k = x(x + 1) \dots (x + n - 1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j + 1)$, $k + 1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n, k) = (n - k)E(n - 1, k - 1) + (k + 1)E(n - 1, k)$$

$$E(n, 0) = E(n, n - 1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n - 1, k - 1) + kS(n - 1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n + 1) \pmod{p}$$

$$B(n) = \sum_{k=0}^n \binom{n}{k} \cdot B(k)$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n - 2)! / ((d_1 - 1)! \dots (d_n - 1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n + 1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n + 1} = \frac{(2n)!}{(n + 1)!n!}$$

$$C_0 = 1, \quad C_{n+1} = \frac{2(2n + 1)}{n + 2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Catalan convolution: find the count of balanced parentheses sequences consisting of $n + k$ pairs of parentheses where the first k symbols are open brackets.

$$C^k = \frac{k + 1}{n + k + 1} \binom{2n + k}{n}$$

6.4 Other

DeBruijnSeq.h

Description: Recursive FKM, given alphabet $[0, k)$ constructs cyclic string of length k^n that contains every length n string as substr. a7faa5, 13 lines

```
vi dseq(int k, int n) {
    if (k == 1) return {0};
    vi res, aux(n+1);
    function<void(int,int)> gen = [&](int t, int p) {
        if (t > n) { // consider lyndon word of len p
            if (n%p == 0) FOR(i, 1, p+1) res.pb(aux[i]);
        } else {
            aux[t] = aux[t-p]; gen(t+1, p);
            FOR(i, aux[t-p]+1, k) aux[t] = i, gen(t+1, t);
        }
    };
    gen(1, 1); return res;
}
```

PermGroup.h

Description: Used only once. Schreier-Sims lets you add a permutation to a group, count number of permutations in a group, and test whether a permutation is a member of a group.

Time: ? 1bebf4, 43 lines

```
int n;
vi inv(vi v) { vi V(sz(v)); FOR(i, sz(v)) V[v[i]]=i; return V; }
vi id() { vi v(n); iota(all(v), 0); return v; }
vi operator*(const vi& a, const vi& b) {
    vi c(sz(a)); FOR(i, sz(a)) c[i] = a[b[i]];
    return c; }
```

```
const int N = 15;
struct Group {
    bool flag[N];
    vi sigma[N]; // sigma[t][k] = t, sigma[t][x] = x if x > k
    vector<vi> gen;
```

```
void clear(int p) {
    memset(flag, 0, sizeof flag);
    flag[p] = 1; sigma[p] = id(); gen.clear();
}
} g[N];
bool check(const vi& cur, int k) {
    if (!k) return 1;
    int t = cur[k];
    return g[k].flag[t] ? check(inv(g[k].sigma[t])*cur, k-1) : 0;
}
void updateX(const vi& cur, int k);
void ins(const vi& cur, int k) {
    if (check(cur, k)) return;
    g[k].gen.pb(cur);
    FOR(i, n) if (g[k].flag[i]) updateX(cur*g[k].sigma[i], k);
}
void updateX(const vi& cur, int k) {
    int t = cur[k]; // if flag, fixes k -> k
    if (g[k].flag[t]) ins(inv(g[k].sigma[t])*cur, k-1);
    else { g[k].flag[t] = 1, g[k].sigma[t] = cur;
           each(x, g[k].gen) updateX(x*cur, k); }
}
ll order(vector<vi> gen) {
    assert(sz(gen)); n = sz(gen[0]); FOR(i, n) g[i].clear(i);
    each(a, gen) ins(a, n-1); // insert perms into group one by one
    ll tot = 1;
    FOR(i, n) {
        int cnt = 0; FOR(j, i+1) cnt += g[i].flag[j];
        tot *= cnt; }
    return tot;
}
```

Graph (7)

7.1 Fundamentals

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.

Time: $\mathcal{O}(|V| + |E|)$ 66a137, 14 lines

```
vi topoSort(const vector<vi>& gr) {
    vi indeg(sz(gr)), ret;
    for (auto& li : gr) for (int x : li) indeg[x]++;
    queue<int> q; // use priority_queue for lexic. largest ans.
    rep(i, 0, sz(gr)) if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int i = q.front(); // top() for priority queue
        ret.push_back(i);
        q.pop();
        for (int x : gr[i])
            if (--indeg[x] == 0) q.push(x);
    }
    return ret;
}
```

7.2 Network flow

MinCostMaxFlow.h

Description: Min-cost max-flow. $\text{cap}[i][j] \neq \text{cap}[j][i]$ is allowed; double edges are not. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: Approximately $\mathcal{O}(E^2)$ fe85cc, 81 lines

```
#include <bits/extc++.h>
```

```
const ll INF = numeric_limits<ll>::max() / 4;
typedef vector<ll> VL;

struct MCMF {
    int N;
    vector<vi> ed, red;
    vector<VL> cap, flow, cost;
    vi seen;
    VL dist, pi;
    vector<pii> par;

    MCMF(int N) :
        N(N), ed(N), red(N), cap(N, VL(N)), flow(cap), cost(cap),
        seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        this->cap[from][to] = cap;
        this->cost[from][to] = cost;
        ed[from].push_back(to);
        red[to].push_back(from);
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});

        auto relax = [&](int i, ll cap, ll cost, int dir) {
            ll val = di - pi[i] + cost;
            if (cap && val < dist[i]) {
                dist[i] = val;
                par[i] = {s, dir};
                if (its[i] == q.end()) its[i] = q.push({-dist[i], i});
                else q.modify(its[i], {-dist[i], i});
            }
        };

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (int i : ed[s]) if (!seen[i])
                relax(i, cap[s][i] - flow[s][i], cost[s][i], 1);
            for (int i : red[s]) if (!seen[i])
                relax(i, flow[i][s], -cost[i][s], 0);
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (int p,r,x = t; tie(p,r) = par[x], x != s; x = p)
                fl = min(fl, r ? cap[p][x] - flow[p][x] : flow[x][p]);
            totflow += fl;
            for (int p,r,x = t; tie(p,r) = par[x], x != s; x = p)
                if (r) flow[p][x] += fl;
                else flow[x][p] -= fl;
        }
        rep(i,0,N) rep(j,0,N) totcost += cost[i][j] * flow[i][j];
        return {totflow, totcost};
    }

    // If some costs can be negative, call this before maxflow:
    void setpi(int s) { // (otherwise, leave this out)
```

```
        fill(all(pi), INF); pi[s] = 0;
        int it = N, ch = 1; ll v;
        while (ch-- && it--)
            rep(i,0,N) if (pi[i] != INF)
                for (int to : ed[i]) if (cap[i][to])
                    if ((v = pi[i] + cost[i][to]) < pi[to])
                        pi[to] = v, ch = 1;
            assert(it >= 0); // negative cost cycle
        }
    };
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $O(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

482fe0, 35 lines

```
template<class T> T edmondsKarp(vector<unordered_map<int, T>>&
    graph, int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vi par(sz(graph)), q = par;

    for (;;) {
        fill(all(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        rep(i,0,ptr) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
    }
out:
    T inc = numeric_limits<T>::max();
    for (int y = sink; y != source; y = par[y])
        inc = min(inc, graph[par[y]][y]);

    flow += inc;
    for (int y = sink; y != source; y = par[y]) {
        int p = par[y];
        if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
        graph[y][p] += inc;
    }
}

}

}

}

}
```

Dinic.h

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max|cap|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{VE})$ for bipartite matching.

d7f0f1, 42 lines

```
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
```

```
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

Time: $\mathcal{O}(V^3)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i,0,n) co[i] = {i};
    rep(ph,1,n) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it,0,n-ph) { //  $O(V^2) \rightarrow O(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i,0,n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i,0,n) mat[s][i] += mat[t][i];
        rep(i,0,n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

Time: $\mathcal{O}(V)$ Flow Computations	
"PushRelabel.h"	0418b3, 13 lines
typedef array<ll, 3> Edge; vector<Edge> gomoryHu(int N, vector<Edge> ed) { vector<Edge> tree; vi par(N); rep(i,1,N) { PushRelabel D(N); <i>// Dinic also works</i> for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]); tree.push_back({i, par[i], D.calc(i, par[i])}); rep(j,i+1,N) if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i; } return tree; }	

7.3 Matching

DFSMatching.h	
Description: Simple bipartite matching algorithm. Graph g should be given as normal adjacency list, no need to specify partitions explicitly Time: $\mathcal{O}(VE)$ worst case but also for $n,m \leq 10^5$ in 0.5s	
	04a846, 28 lines

struct matching { vector<vi> v; int n; vi odw, skoj; matching(vector <vi> &vec) : v(vec), n(sz(vec)), odw(n, 0), skoj(n, -1) {} bool dfs(int x) { if (odw[x]) return 0; odw[x] = 1; trav(u, v[x]) { if (skoj[u] == -1 dfs(skoj[u])) { skoj[u] = x; skoj[x] = u; return 1; } } return 0; } int solve() { int ok = 1, res = 0; while (ok--) { fill(all(odw), 0); rep(i, 0, n) { if (skoj[i] == -1 && dfs(i)) res++, ok = 1; } return res; } } };	
---	--

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set. Left vertices are [0,n), right: [n,n+m) (!!!!)	
"DFSMatching.h"	1b9ec1, 25 lines

vi cover(vector<vi>& g, int n, int m) { matching solver(g); int res = solver.solve(); vector< bool > lfound(n, true), seen(m); vi q, cover; rep(i,0,n) { if (solver.skoj[i] != -1) lfound[i] = false ; } rep(i,0,n) { if (lfound[i]) q.push_back(i); } }	
--	--

while (!q.empty()) { int i = q.back(); q.pop_back(); assert(0 <= i && i < n); lfound[i] = 1; trav(e, g[i]) if (!seen[e-n] && solver.skoj[e] != -1) { seen[e-n] = true ; q.push_back(solver.skoj[e]); } } rep(i,0,n) if (!lfound[i]) cover.push_back(i); rep(i,0,m) if (seen[i]) cover.push_back(n+i); assert(sz(cover) == res); return cover; }	
---	--

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Time: $\mathcal{O}(N^2M)$	
	1e0fe9, 31 lines

pair< int , vi> hungarian(const vector<vi> &a) { if (a.empty()) return {0, {}}; int n = sz(a) + 1, m = sz(a[0]) + 1; vi u(n), v(m), p(m), ans(n - 1); rep(i,1,n) { p[0] = i; int j0 = 0; <i>// add "dummy" worker 0</i> vi dist(m, INT_MAX), pre(m, -1); vector< bool > done(m + 1); do { <i>// dijkstra</i> done[j0] = true ; int i0 = p[j0], j1, delta = INT_MAX; rep(j,1,m) if (!done[j]) { auto cur = a[i0 - 1][j - 1] - u[i0] - v[j]; if (cur < dist[j]) dist[j] = cur, pre[j] = j0; if (dist[j] < delta) delta = dist[j], j1 = j; } rep(j,0,m) { if (done[j]) u[p[j]] += delta, v[j] -= delta; else dist[j] -= delta; } j0 = j1; } while (p[j0]); while (j0) { <i>// update alternating path</i> int j1 = pre[j0]; p[j0] = p[j1], j0 = j1; } } rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1; return {-v[0], ans}; <i>// min cost</i> }	
--	--

GeneralMatching.h

Description: Matching for general graphs using Blossom algorithm. Time: $\mathcal{O}(NM)$ surprisingly fast in practice	
	3f5cfa, 52 lines

vi Blossom(vector<vi> &graph) { int n = sz(graph), timer = -1; vi mate(n, -1), label(n), parent(n), orig(n), aux(n, -1), q; auto lca = [&](int x, int y) { for (timer++; ; swap(x, y)) { if (x == -1) continue ; if (aux[x] == timer) return x; aux[x] = timer; x = (mate[x] == -1 ? -1 : orig[parent[mate[x]]]); } } while (n > 1) { int i = 0; while (i < n) { if (i % 2 == 0) { int v = i; while (v < n) { if (v % 2 == 0) { if (v == i) continue; if (v == -1) continue; if (v == -2) continue; if (v == -3) continue; if (v == -4) continue; if (v == -5) continue; if (v == -6) continue; if (v == -7) continue; if (v == -8) continue; if (v == -9) continue; if (v == -10) continue; if (v == -11) continue; if (v == -12) continue; if (v == -13) continue; if (v == -14) continue; if (v == -15) continue; if (v == -16) continue; if (v == -17) continue; if (v == -18) continue; if (v == -19) continue; if (v == -20) continue; if (v == -21) continue; if (v == -22) continue; if (v == -23) continue; if (v == -24) continue; if (v == -25) continue; if (v == -26) continue; if (v == -27) continue; if (v == -28) continue; if (v == -29) continue; if (v == -30) continue; if (v == -31) continue; if (v == -32) continue; if (v == -33) continue; if (v == -34) continue; if (v == -35) continue; if (v == -36) continue; if (v == -37) continue; if (v == -38) continue; if (v == -39) continue; if (v == -40) continue; if (v == -41) continue; if (v == -42) continue; if (v == -43) continue; if (v == -44) continue; if (v == -45) continue; if (v == -46) continue; if (v == -47) continue; if (v == -48) continue; if (v == -49) continue; if (v == -50) continue; if (v == -51) continue; if (v == -52) continue; if (v == -53) continue; if (v == -54) continue; if (v == -55) continue; if (v == -56) continue; if (v == -57) continue; if (v == -58) continue; if (v == -59) continue; if (v == -60) continue; if (v == -61) continue; if (v == -62) continue; if (v == -63) continue; if (v == -64) continue; if (v == -65) continue; if (v == -66) continue; if (v == -67) continue; if (v == -68) continue; if (v == -69) continue; if (v == -70) continue; if (v == -71) continue; if (v == -72) continue; if (v == -73) continue; if (v == -74) continue; if (v == -75) continue; if (v == -76) continue; if (v == -77) continue; if (v == -78) continue; if (v == -79) continue; if (v == -80) continue; if (v == -81) continue; if (v == -82) continue; if (v == -83) continue; if (v == -84) continue; if (v == -85) continue; if (v == -86) continue; if (v == -87) continue; if (v == -88) continue; if (v == -89) continue; if (v == -90) continue; if (v == -91) continue; if (v == -92) continue; if (v == -93) continue; if (v == -94) continue; if (v == -95) continue; if (v == -96) continue; if (v == -97) continue; if (v == -98) continue; if (v == -99) continue; if (v == -100) continue; if (v == -101) continue; if (v == -102) continue; if (v == -103) continue; if (v == -104) continue; if (v == -105) continue; if (v == -106) continue; if (v == -107) continue; if (v == -108) continue; if (v == -109) continue; if (v == -110) continue; if (v == -111) continue; if (v == -112) continue; if (v == -113) continue; if (v == -114) continue; if (v == -115) continue; if (v == -116) continue; if (v == -117) continue; if (v == -118) continue; if (v == -119) continue; if (v == -120) continue; if (v == -121) continue; if (v == -122) continue; if (v == -123) continue; if (v == -124) continue; if (v == -125) continue; if (v == -126) continue; if (v == -127) continue; if (v == -128) continue; if (v == -129) continue; if (v == -130) continue; if (v == -131) continue; if (v == -132) continue; if (v == -133) continue; if (v == -134) continue; if (v == -135) continue; if (v == -136) continue; if (v == -137) continue; if (v == -138) continue; if (v == -139) continue; if (v == -140) continue; if (v == -141) continue; if (v == -142) continue; if (v == -143) continue; if (v == -144) continue; if (v == -145) continue; if (v == -146) continue; if (v == -147) continue; if (v == -148) continue; if (v == -149) continue; if (v == -150) continue; if (v == -151) continue; if (v == -152) continue; if (v == -153) continue; if (v == -154) continue; if (v == -155) continue; if (v == -156) continue; if (v == -157) continue; if (v == -158) continue; if (v == -159) continue; if (v == -160) continue; if (v == -161) continue; if (v == -162) continue; if (v == -163) continue; if (v == -164) continue; if (v == -165) continue; if (v == -166) continue; if (v == -167) continue; if (v == -168) continue; if (v == -169) continue; if (v == -170) continue; if (v == -171) continue; if (v == -172) continue; if (v == -173) continue; if (v == -174) continue; if (v == -175) continue; if (v == -176) continue; if (v == -177) continue; if (v == -178) continue; if (v == -179) continue; if (v == -180) continue; if (v == -181) continue; if (v == -182) continue; if (v == -183) continue; if (v == -184) continue; if (v == -185) continue; if (v == -186) continue; if (v == -187) continue; if (v == -188) continue; if (v == -189) continue; if (v == -190) continue; if (v == -191) continue; if (v == -192) continue; if (v == -193) continue; if (v == -194) continue; if (v == -195) continue; if (v == -196) continue; if (v == -197) continue; if (v == -198) continue; if (v == -199) continue; if (v == -200) continue; if (v == -201) continue; if (v == -202) continue; if (v == -203) continue; if (v == -204) continue; if (v == -205) continue; if (v == -206) continue; if (v == -207) continue; if (v == -208) continue; if (v == -209) continue; if (v == -210) continue; if (v == -211) continue; if (v == -212) continue; if (v == -213) continue; if (v == -214) continue; if (v == -215) continue; if (v == -216) continue; if (v == -217) continue; if (v == -218) continue; if (v == -219) continue; if (v == -220) continue; if (v == -221) continue; if (v == -222) continue; if (v == -223) continue; if (v == -224) continue; if (v == -225) continue; if (v == -226) continue; if (v == -227) continue; if (v == -228) continue; if (v == -229) continue; if (v == -230) continue; if (v == -231) continue; if (v == -232) continue; if (v == -233) continue; if (v == -234) continue; if (v == -235) continue; if (v == -236) continue; if (v == -237) continue; if (v == -238) continue; if (v == -239) continue; if (v == -240) continue; if (v == -241) continue; if (v == -242) continue; if (v == -243) continue; if (v == -244) continue; if (v == -245) continue; if (v == -246) continue; if (v == -247) continue; if (v == -248) continue; if (v == -249) continue; if (v == -250) continue; if (v == -251) continue; if (v == -252) continue; if (v == -253) continue; if (v == -254) continue; if (v == -255) continue; if (v == -256) continue; if (v == -257) continue; if (v == -258) continue; if (v == -259) continue; if (v == -260) continue; if (v == -261) continue; if (v == -262) continue; if (v == -263) continue; if (v == -264) continue; if (v == -265) continue; if (v == -266) continue; if (v == -267) continue; if (v == -268) continue; if (v == -269) continue; if (v == -270) continue; if (v == -271) continue; if (v == -272) continue; if (v == -273) continue; if (v == -274) continue; if (v == -275) continue; if (v == -276) continue; if (v == -277) continue; if (v == -278) continue; if (v == -279) continue; if (v == -280) continue; if (v == -281) continue; if (v == -282) continue; if (v == -283) continue; if (v == -284) continue; if (v == -285) continue; if (v == -286) continue; if (v == -287) continue; if (v == -288) continue; if (v == -289) continue; if (v == -290) continue; if (v == -291) continue; if (v == -292) continue; if (v == -293) continue; if (v == -294) continue; if (v == -295) continue; if (v == -296) continue; if (v == -297) continue; if (v == -298) continue; if (v == -299) continue; if (v == -300) continue; if (v == -301) continue; if (v == -302) continue; if (v == -303) continue; if (v == -304) continue; if (v == -305) continue; if (v == -306) continue; if (v == -307) continue; if (v == -308) continue; if (v == -309) continue; if (v == -310) continue; if (v == -311) continue; if (v == -312) continue; if (v == -313) continue; if (v == -314) continue; if (v == -315) continue; if (v == -316) continue; if (v == -317) continue; if (v == -318) continue; if (v == -319) continue; if (v == -320) continue; if (v == -321) continue; if (v == -322) continue; if (v == -323) continue; if (v == -324) continue; if (v == -325) continue; if (v == -326) continue; if (v == -327) continue; if (v == -328) continue; if (v == -329) continue; if (v == -330) continue; if (v == -331) continue; if (v == -332) continue; if (v == -333) continue; if (v == -334) continue; if (v == -335) continue; if (v == -336) continue; if (v == -337) continue; if (v == -338) continue; if (v == -339) continue; if (v == -340) continue; if (v == -341) continue; if (v == -342) continue; if (v == -343) continue; if (v == -344) continue; if (v == -345) continue; if (v == -346) continue; if (v == -347) continue; if (v == -348) continue; if (v == -349) continue; if (v == -350) continue; if (v == -351) continue; if (v == -352) continue; if (v == -353) continue; if (v == -354) continue; if (v == -355) continue; if (v == -356) continue; if (v == -357) continue; if (v == -358) continue; if (v == -359) continue; if (v == -360) continue; if (v == -361) continue; if (v == -362) continue; if (v == -363) continue; if (v == -364) continue; if (v == -365) continue; if (v == -366) continue; if (v == -367) continue; if (v == -368) continue; if (v == -369) continue; if (v == -370) continue; if (v == -371) continue; if (v == -372) continue; if (v == -373) continue; if (v == -374) continue; if (v == -375) continue; if (v == -376) continue; if (v == -377) continue; if (v == -378) continue; if (v == -379) continue; if (v == -380) continue; if (v == -381) continue; if (v == -382) continue; if (v == -383) continue; if (v == -384) continue; if (v == -385) continue; if (v == -386) continue; if (v == -387) continue; if (v == -388) continue; if (v == -389) continue; if (v == -390) continue; if (v == -391) continue; if (v == -392) continue; if (v == -393) continue; if (v == -394) continue; if (v == -395) continue; if (v == -396) continue; if (v == -397) continue; if (v == -398) continue; if (v == -399) continue; if (v == -400) continue; if (v == -401) continue; if (v == -402) continue; if (v == -403) continue; if (v == -404) continue; if (v == -405) continue; if (v == -406) continue; if (v == -407) continue; if (v == -408) continue; if (v == -409) continue; if (v == -410) continue; if (v == -411) continue; if (v == -412) continue; if (v == -413) continue; if (v == -414) continue; if (v == -415) continue; if (v == -416) continue; if (v == -417) continue; if (v == -418) continue; if (v == -419) continue; if (v == -420) continue; if (v == -421) continue; if (v == -422) continue; if (v == -423) continue; if (v == -424) continue; if (v == -425) continue; if (v == -426) continue; if (v == -427) continue; if (v == -428) continue; if (v == -429) continue; if (v == -430) continue; if (v == -431) continue; if (v == -432) continue; if (v == -433) continue; if (v == -434) continue; if (v == -435) continue; if (v == -436) continue; if (v == -437) continue; if (v == -438) continue; if (v == -439) continue; if (v == -440) continue; if (v == -441) continue; if (v == -442) continue; if (v == -443) continue; if (v == -444) continue; if (v == -445) continue; if (v == -446) continue; if (v == -447) continue; if (v == -448) continue; if (v == -449) continue; if (v == -450) continue; if (v == -451) continue; if (v == -452) continue; if (v == -453) continue; if (v == -454) continue; if (v == -455) continue; if (v == -456) continue; if (v == -457) continue; if (v == -458) continue; if (v == -459) continue; if (v == -460) continue; if (v == -461) continue; if (v == -462) continue; if (v == -463) continue; if (v == -464) continue; if (v == -465) continue; if (v == -466) continue;<

```
    }
    return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

DominatorTree.h

Description: Calculates the dominator tree of a directed graph with given source. ans[i] is the immediate dominator of vertex i, ans[i] = i means vertex unreachable or source
Time: $\mathcal{O}(N\alpha)$

7e2b88, 60 lines

```
struct dominator_tree {
    vector<basic_string<int>> g, rg, bucket;
    vi arr, par, rev, sdom, dom, dsu, label;
    int n, t;
    dominator_tree(int n) : g(n), rg(n), bucket(n), arr(n, -1),
        par(n), rev(n), sdom(n), dom(n), dsu(n), label(n), n(n), t
            (0) {}
    void add_edge(int u, int v) { g[u] += v; }
    void dfs(int u) {
        arr[u] = t;
        rev[t] = u;
        label[t] = sdom[t] = dsu[t] = t;
        t++;
        for (int w : g[u]) {
            if (arr[w] == -1) {
                dfs(w);
                par[arr[w]] = arr[u];
            }
            rg[arr[w]] += arr[u];
        }
    }
    int find(int u, int x=0) {
        if (u == dsu[u])
            return x ? -1 : u;
        int v = find(dsu[u], x+1);
        if (v < 0)
            return u;
        if (sdom[label[dsu[u]]] < sdom[label[u]])
            label[u] = label[dsu[u]];
        dsu[u] = v;
        return x ? v : label[u];
    }
    vi solve(int root) {
        dfs(root);
        iota(all(dom), 0);
        for (int i=t-1; i>=0; i--) {
            for (int w : rg[i])
                sdom[i] = min(sdom[i], sdom[find(w)]);
            if (i)
                bucket[sdom[i]] += i;
            for (int w : bucket[i]) {
                int v = find(w);
                if (sdom[v] == sdom[w])
                    dom[w] = sdom[w];
                else
                    dom[w] = v;
            }
            if (i > 1)
                dsu[i] = par[i];
        }
        for (int i=1; i<t; i++) {
            if (dom[i] != sdom[i])
                dom[i] = dom[dom[i]];
        }
    }
};
```

```
    }
    vi outside_dom(n);
    iota(all(outside_dom), 0);
    for (int i=0; i<t; i++)
        outside_dom[rev[i]] = rev[dom[i]];
    return outside_dom;
}
};
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps([&](const vi& edgelist) {...});
Time: $\mathcal{O}(E+V)$

2965e5, 33 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
    int me = num[at] = ++Time, e, y, top = me;
    for (auto pa : ed[at]) if (pa.second != par) {
        tie(y, e) = pa;
        if (num[y]) {
            top = min(top, num[y]);
            if (num[y] < me)
                st.push_back(e);
        } else {
            int si = sz(st);
            int up = dfs(y, e, f);
            top = min(top, up);
            if (up == me) {
                st.push_back(e);
                f(vi(st.begin() + si, st.end()));
                st.resize(si);
            }
            else if (up < me) st.push_back(e);
            else { /* e is a bridge */ }
        }
    }
    return top;
}

template<class F>
void bicomps(F f) {
    num.assign(sz(ed), 0);
    rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a \vee b) \wedge (\neg a \vee c) \wedge (d \vee \neg b) \wedge \dots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions ($\neg x$).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0, ~1, 2}); // ≤ 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars

Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }

    void setValue(int x) { either(x, x); }

    void atMostOne(const vi& li) { // (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        rep(i,2,sz(li)) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ? dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        rep(i,0,2*N) if (!comp[i]) dfs(i);
        rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
    int n = sz(gr);
```



```
vi D(n), its(n), eu(nedges), ret, s = {src};
D[src]++; // to allow Euler paths, not just cycles
while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
        D[x]--, D[y]++;
        eu[e] = 1; s.push_back(y);
    }
}
for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
return {ret.rbegin(), ret.rend()};
}
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int& z = free[y] = 0; adj[y][z] != -1; z++);
    }
    rep(i,0,sz(eds))
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
    return ret;
}
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
    if (!P.any()) { if (!X.any()) f(R); return; }
    auto q = (P | X)._FindFirst();
    auto cands = P & ~eds[q];
    rep(i,0,sz(eds)) if (cands[i]) {
```

```
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax)) return;
            q.push_back(R.back().i);
            vv T;
            for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk + 1].clear();
                    if (k < mnk) T[j++].i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                rep(k,mnk,mxk + 1) for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (sz(q) > sz(qmax)) qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
    vi maxClique() { init(V), expand(V); return qmax; }
    Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
        rep(i,0,sz(e)) V.push_back({i});
    }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
    int on = 1, d = 1;
    while (on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    rep(i,1,d) rep(j,0,sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
    rep(i,0,sz(tbl))
        if (steps&(1<<i)) nod = tbl[i][nod];
    return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

../data-structures/RMQ.h"7c2078, 19 lines

```
struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;
    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }
    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    int dist(int a, int b){return time[a] + time[b] - 2*time[lca(a,b)];}
};
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, node) representing a tree. No index reordering occurs. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h"a8ba0a, 19 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
```

```

vi li = subset, &T = lca.time;
auto cmp = [&](int a, int b) { return T[a] < T[b]; };
sort(all(li), cmp);
int m = sz(li)-1;
rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
}
sort(all(li), cmp);
li.erase(unique(all(li)), li.end());
vpi ret = {{li[0], li[0]}};
rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(lca.lca(a, b), b);
}
return ret;
}

```

Centroid.h

Description: Centroid decomposition template.

c6f439, 73 lines

```

// override calc() to do what you need
// node indexes are preserved throughout invocations
// if something is protected, it is safe to use it in calc()
struct base_centroids {
private:
    int n; //graph size
    vector<vi> v; //0-based
    vector<bool> odw, gold;
    vi sub, maxsub;
protected:
    vi par; //current array of nodes' parents. par[root]=-1.
    vi get_subtrees(vi &pre) { //helper fn which finds ranges
        [, ) of root's subtrees.
        vi res = {};
        rep(i, 0, sz(pre)) {
            if (par[pre[i]] == pre[0]) res.push_back(i);
        }
        res.push_back(sz(pre));
        return res;
    }
}
//calculate answers for the current centroid(root). Nodes
given in PREORDER
virtual void calc(int root, vi &nodes) = 0;
private:
void prep(int x, vi &nodes) {
    odw[x] = 1; sub[x] = 1;
    nodes.push_back(x);
    trav(u, v[x]) {
        if (!gold[u] && !odw[u]) {
            prep(u, nodes);
            sub[x] += sub[u];
            maxsub[x] = max(maxsub[x], sub[u]);
        }
    }
}
void cendfs(int x, int &PRE, vi &pre) {
    odw[x] = 1; pre[PRE++] = x;
    trav(u, v[x]) {
        if (!odw[u] && !gold[u]) {
            par[u] = x;
            cendfs(u, PRE, pre);
        }
    }
}
public:
base_centroids(int N, vector<vi> graph) : n(N), v(graph),
    odw(n, false), gold(n, false), sub(n, 0), maxsub(n, 0),
    par(n, -1) {

```

```

};
void solve(int start=0) {
    vector<int> comp;
    prep(start, comp);
    int N = sz(comp), cen = -1;
    trav(node, comp) {
        maxsub[node] = max(maxsub[node], N - sub[node]);
        if (maxsub[node] <= N / 2) cen = node;
        odw[node] = 0, sub[node] = 0, maxsub[node] = 0;
    }
    int PRE = 0;
    vi pre(N, 0);
    par[cen] = -1;
    cendfs(cen, PRE, pre);
    calc(cen, pre);
    trav(node, comp) odw[node] = 0, par[node] = -1;
    gold[cen] = 1;
    trav(u, v[cen]) {
        if (!gold[u]) solve(u);
    }
}
}
};

struct centroids : base_centroids {
    centroids(int N, vector<vi> graph) : base_centroids(N,
        graph) {}
    void calc(int root, vi &nodes) {
        trav(node, nodes) cerr << node << "\n";
    }
}
};

```

HLD.h

Description: Handles subtree and path queries simultaneously in one lazy_segtree. Each subtree is 1 segment, while path is $O(\log N)$ segments in the tree. *VALS_EDGES* being true means that values are stored in the edges, as opposed to the nodes. All values are initialized to the segtree default.

Time: $\mathcal{O}((\log N)^2)$, one logarithm for subtrees.

../data-structures/LazySegmentTree.h 35c6c4, 56 lines

```

template<bool VALS_EDGES,
class S, S(*op)(S, S), S(*e)(),
class F, S (*mapping)(F, S), F(*composition)(F, F), F (*id)()>
struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, depth, rt, pos;
    lazy_segtree<S, op, e, F, mapping, composition, id> tree;
    HLD(vector<vi> adj_, int root=0) :
        N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1), depth(N),
        rt(N, root), pos(N), tree(vector<S>(N, e())) {dfsSz(root),
            dfsHld(root);}
    void dfsSz(int v) {
        if (par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
        trav(u, adj[v]){
            par[u] = v, depth[u] = depth[v] + 1;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        trav(u, adj[v]){
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
}
template<class B> int process(int u, int v, B query) {
    for (; rt[u] != rt[v]; v = par[rt[v]]) {

```

```

        if (depth[rt[u]] > depth[rt[v]]) swap(u, v);
        query(pos[rt[v]], pos[v] + 1);
    }
    if (depth[u] > depth[v]) swap(u, v);
    query(pos[u] + VALS_EDGES, pos[v] + 1);
    return u;
}
int lca(int u, int v) {
    return process(u, v, [](int l, int r) {});
}
void path_apply(int u, int v, F func) {
    process(u, v, [&](int l, int r) {tree.apply(l, r, func);})
    ;
}
S path_prod(int u, int v) {
    S res = e();
    process(u, v, [&](int l, int r) {
        res = op(res, tree.prod(l, r));
    });
    return res;
}
void subtree_apply(int v, F func) {
    tree.apply(pos[v] + VALS_EDGES, pos[v] + siz[v], func);
}
S subtree_prod(int v) {
    return tree.prod(pos[v] + VALS_EDGES, pos[v] + siz[v]);
}
};

```

LinkCutTree.h

Description: A dynamic data structure for rooted trees. 1-based(!!!) Path queries need commutativity + neutral element. Subtree queries need that + existence of inverse elements Lazy propagation is possible, too.

Time: All operations take amortized $\mathcal{O}(\log N)$.

161911, 91 lines

```

template<class T, T(*op)(T, T), T(*inv)(T), T(*e)()>
struct LCT {
    struct node {
        int ch[2] = {0, 0}, p = 0;
        T self = e(), path = e(); // Path aggregates
        T sub = e(), vir = e(); // Subtree aggregates
        bool flip = 0; // Lazy tags
    };
    vector<node> t;
    LCT(int n) : t(n + 1) {}
    void push(int x) {
        if (!x || !t[x].flip) return;
        int l = t[x].ch[0], r = t[x].ch[1];
        t[l].flip ^= 1, t[r].flip ^= 1;
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].flip = 0;
    }
    void pull(int x) {
        int l = t[x].ch[0], r = t[x].ch[1]; push(l); push(r);

        t[x].path = op(op(t[l].path, t[x].self), t[r].path);
        t[x].sub = op(op(op(t[x].vir, t[l].sub), t[r].sub), t[x].
            self);
    }
    void set(int x, int d, int y) {
        t[x].ch[d] = y; t[y].p = x; pull(x);
    }
    void splay(int x) {
        auto dir = [&](int x) {
            int p = t[x].p; if (!p) return -1;
            return t[p].ch[0] == x ? 0 : t[p].ch[1] == x ? 1 : -1;
        };
        auto rotate = [&](int x) {
            int y = t[x].p, z = t[y].p, dx = dir(x), dy = dir(y);

```

```
set(y, dx, t[x].ch[!dx]);
set(x, !dx, y);
if (~dy) set(z, dy, x);
t[x].p = z;
};
for (push(x); ~dir(x); ) {
    int y = t[x].p, z = t[y].p;
    push(z); push(y); push(x);
    int dx = dir(x), dy = dir(y);
    if (~dy) rotate(dx != dy ? x : y);
    rotate(x);
}
}
int access(int x) {
    int u = x, v = 0;
    for (; u; v = u, u = t[u].p) {
        splay(u);
        int& ov = t[u].ch[1];
        t[u].vir = op(t[u].vir, t[ov].sub);
        t[u].vir = op(t[u].vir, inv(t[v].sub));
        ov = v; pull(u);
    }
    return splay(x), v;
}
void reroot(int x) {
    access(x); t[x].flip ^= 1; push(x);
}
void link(int u, int v) {
    reroot(u); access(v);
    t[v].vir = op(t[v].vir, t[u].sub);
    t[u].p = v; pull(v);
}
void cut(int u, int v) {
    reroot(u); access(v);
    t[v].ch[0] = t[u].p = 0; pull(v);
}
// Rooted tree LCA. Returns 0 if u and v arent connected.
int lca(int u, int v) {
    if (u == v) return u;
    access(u); int ret = access(v);
    return t[u].p ? ret : 0;
}
// Query subtree of u where v is outside the subtree.
T subtree_prod(int u, int v) {
    reroot(v); access(u); return op(t[u].vir, t[u].self);
}
// Query path [u..v]
T path_prod(int u, int v) {
    reroot(u); access(v); return t[v].path;
}
// Update vertex u with value v
void set(int u, T v) {
    access(u); t[u].self = v; pull(u);
}
T get(int u) {
    return t[u].self;
}
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

```

"../data-structures/UnionFindRollback.h" 39e620, 60 lines
struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
};
```

DirectedMST Point lineDistance SegmentDistance

```
void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
}
Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1,-1}), comp;
    deque<tuple<int, int, vector<Edge>>> cyps;
    rep(s,0,n) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1,{};};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cyps.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
    }

    for (auto& [u,t,comp] : cyps) { // restore sol (optional)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto& e : comp) in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    rep(i,0,n) par[i] = in[i].a;
    return {res, par};
}
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $mat[a][b]--$, $mat[b][b]++$ (and $mat[b][a]--$, $mat[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

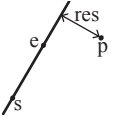
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```

47ec0a, 28 lines
template<class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << "," << p.y << ")"; }
};
```

lineDistance.h

Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

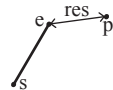


```

"f6bf6b, 4 lines"
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double)(b-a).cross(p-a) / (b-a).dist();
}
```

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.



Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

"Point.h"	5c88f4, 6 lines
-----------	-----------------

```

typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}

```

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
"Point.h", "OnSegment.h"

9d57f2, 13 lines

```

template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}

```

lineIntersection.h

Description:

If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
"Point.h"

a01f81, 8 lines

```

template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}

```

sideOf.h

Description: Returns where *p* is as seen from *s* towards *e*. 1/0/-1 ⇔ left/on line/right. If the optional argument *eps* is given 0 is returned if *p* is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

"Point.h"	3af81c, 9 lines
-----------	-----------------

```

template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}

```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

c597e8, 3 lines

```

template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}

```

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

03a306, 6 lines

```

typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}

```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

0f0602, 35 lines

```

struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (1l)b.x) <
        make_tuple(b.t, b.half(), a.x * (1l)b.y);
}

```

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
 if (b < a) swap(a, b);
 return (b < a.t180() ?
 make_pair(a, b) : make_pair(b, a.t360()));
}

Angle **operator**+(Angle a, Angle b) { // point a + vector b
 Angle r(a.x + b.x, a.y + b.y, a.t);
 if (a.t180() < r) r.t--;
 return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) { // angle b - angle a
 int tu = b.t - a.t; a.t = b.t;
 return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

84d6d3, 11 lines

```

typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}

```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

b0153d, 13 lines

```

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}

```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

a1ee63, 19 lines

```

"../../content/geometry/Point.h"
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;

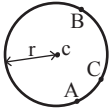
```

```
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
};
auto sum = 0.0;
rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
return sum;
}
```

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



```
"Point.h" 1caa3a, 9 lines

typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()* (C-B).dist()* (A-C).dist() /
        abs((B-A).cross(C-A))/2;
}
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

```
"circumcircle.h" 09dd0a, 17 lines

pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};

bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

```
"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
"Point.h" f12300, 6 lines

template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

```
"Point.h" 9706dc, 9 lines

typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h

Description:

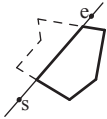
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;

p = polygonCut(p, P(0,0), P(1,0));

```
"Point.h", "lineIntersection.h" f2b7d4, 13 lines

typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;
    rep(i,0,sz(poly)) {
        P cur = poly[i], prev = i ? poly[i-1] : poly.back();
        bool side = s.cross(e, cur) < 0;
        if (side != (s.cross(e, prev) < 0))
            res.push_back(lineInter(s, e, cur, prev).second);
        if (side)
            res.push_back(cur);
    }
    return res;
}
```



ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" 310954, 13 lines

typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```



HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

```
"Point.h" c571b8, 12 lines

typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j)
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

```
"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

```
"Point.h" 7cf45b, 39 lines

#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
}
```

```

    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i,0,2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}

```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(*lo - p).dist2(), { *lo, p }});
        S.insert(p);
    }
    return ret.second;
}

```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h" bac5b0, 63 lines

```

typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

```

```

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }

```

```

struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(const P& p) { // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x,y) - p).dist2();
    }
}

```

```

Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
        x0 = min(x0, p.x); x1 = max(x1, p.x);

```

```

        y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
        // split on x if width >= height (not ideal...)
        sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
        // divide by taking half the array for each child (not
        // best performance with many duplicates in the middle)
        int half = sz(vp)/2;
        first = new Node({vp.begin(), vp.begin() + half});
        second = new Node({vp.begin() + half, vp.end()});
    }
}
}
};

```

```

struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}
}

```

```

pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
        // uncomment if we should not find the point itself:
        // if (p == node->pt) return {INF, P()};
        return make_pair((p - node->pt).dist2(), node->pt);
    }
}

```

```

Node *f = node->first, *s = node->second;
T bfirst = f->distance(p), bsec = s->distance(p);
if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

```

```

// search closest side first, other side if needed
auto best = search(f, p);
if (bsec < best.first)
    best = min(best, search(s, p));
return best;
}

```

```

// find nearest point to a point, and its squared distance
// (requires an arbitrary operator< for Point)
pair<T, P> nearest(const P& p) {
    return search(root, p);
}
}

```

};

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 88 lines

```

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

```

```

struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

```

```

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}

```

```

Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}

void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

```

```

pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }
}

```

```

#define H(e) e->F(), e->p
#define valid(e) (e->F()).cross(H(base)) > 0)
Q A, B, ra, rb;
int half = sz(s) / 2;
tie(ra, A) = rec({all(s) - half});
tie(B, rb) = rec({sz(s) - half + all(s)});
while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
Q base = connect(B->r(), A);
if (A->p == ra->p) ra = base->r();
if (B->p == rb->p) rb = base;

```

```

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
for (;;) {
    DEL(LC, base->r(), o); DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
        base = connect(RC, base->r());
    else
        base = connect(base->r(), LC->r());
}
return { ra, rb };
}

```

```

vector<P> triangulate(vector<P> pts) {
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
    #define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
        q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
    return pts;
}

```

<pre> } </pre>	3058c3, 6 lines
8.5 3D PolyhedronVolume.h Description: Magic formula for the volume of a polyhedron. Faces should point outwards.	

<pre> template<class V, class L> double signedPolyVolume(const V& p, const L& trilst) { double v = 0; for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]); return v / 6; } </pre>	
---	--

Point3D.h Description: Class to handle points in 3D space. T can be e.g. double or long long.	8058ae, 32 lines
--	------------------

<pre> template<class T> struct Point3D { typedef Point3D P; typedef const P& R; T x, y, z; explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {} bool operator<(R p) const { return tie(x, y, z) < tie(p.x, p.y, p.z); } bool operator==(R p) const { return tie(x, y, z) == tie(p.x, p.y, p.z); } P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); } P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); } P operator*(T d) const { return P(x*d, y*d, z*d); } P operator/(T d) const { return P(x/d, y/d, z/d); } T dot(R p) const { return x*p.x + y*p.y + z*p.z; } P cross(R p) const { return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x); } T dist2() const { return x*x + y*y + z*z; } double dist() const { return sqrt((double)dist2()); } //Azimuthal angle (longitude) to x-axis in interval [-pi, pi] double phi() const { return atan2(y, x); } //Zenith angle (latitude) to the z-axis in interval [0, pi] double theta() const { return atan2(sqrt(x*x+y*y),z); } P unit() const { return *this/(T)dist(); } //makes dist()=1 //returns unit vector normal to *this and p P normal(P p) const { return cross(p).unit(); } //returns point rotated 'angle' radians ccw around axis P rotate(double angle, P axis) const { double s = sin(angle), c = cos(angle); P u = axis.unit(); return u*dot(u)*(1-c) + (*this)*c - cross(u)*s; } }; </pre>	
---	--

3dHull.h Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards. Time: $\mathcal{O}(n^2)$	5b45fc, 49 lines
--	------------------

<pre> typedef Point3D<double> P3; struct PR { void ins(int x) { (a == -1 ? a : b) = x; } void rem(int x) { (a == x ? a : b) = -1; } int cnt() { return (a != -1) + (b != -1); } int a, b; }; </pre>	
--	--

<pre> struct F { P3 q; int a, b, c; }; </pre>	
---	--

<pre> vector<F> hull3d(const vector<P3>& A) { assert(sz(A) >= 4); vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1})); #define E(x,y) E[f.x][f.y] vector<F> FS; auto mf = [&](int i, int j, int k, int l) { P3 q = (A[j] - A[i]).cross((A[k] - A[i])); if (q.dot(A[l]) > q.dot(A[i])) q = q * -1; F f{q, i, j, k}; E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i); FS.push_back(f); }; rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4) mf(i, j, k, 6 - i - j - k); rep(i,4,sz(A)) { rep(j,0,sz(FS)) { F f = FS[j]; if (f.q.dot(A[i]) > f.q.dot(A[f.a])) { E(a,b).rem(f.c); E(a,c).rem(f.b); E(b,c).rem(f.a); swap(FS[j--], FS.back()); FS.pop_back(); } int nw = sz(FS); rep(j,0,nw) { F f = FS[j]; #define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c); C(a, b, c); C(a, c, b); C(b, c, a); } for (F& it : FS) if ((A[it.b] - A[it.a]).cross(A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b); return FS; }; }; } </pre>	611f07, 8 lines
--	-----------------

sphericalDistance.h Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.	
--	--

<pre> double sphericalDistance(double f1, double t1, double f2, double t2, double radius) { double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1); double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1); double dz = cos(t2) - cos(t1); double d = sqrt(dx*dx + dy*dy + dz*dz); return radius*2*asin(d/2); } </pre>	
--	--

Strings (9)

KMP.h Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string. Time: $\mathcal{O}(n)$	d4375c, 16 lines
---	------------------

<pre> vi pi(const string& s) { vi p(sz(s)); </pre>	
--	--

<pre> rep(i,1,sz(s)) { int g = p[i-1]; while (g && s[i] != s[g]) g = p[g-1]; p[i] = g + (s[i] == s[g]); } return p; } </pre>	
--	--

<pre> vi match(const string& s, const string& pat) { vi p = pi(pat + '\0' + s), res; rep(i,sz(p)-sz(s),sz(p)) if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat)); return res; } </pre>	
---	--

Zfunc.h Description: z[x] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301) Time: $\mathcal{O}(n)$	3ae526, 12 lines
--	------------------

<pre> vi Z(string S) { vi z(sz(S)); int l = -1, r = -1; rep(i,1,sz(S)) { z[i] = i >= r ? 0 : min(r - i, z[i - l]); while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]]) z[i]++; if (i + z[i] > r) l = i, r = i + z[i]; } return z; } </pre>	
--	--

Manacher.h Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down). Time: $\mathcal{O}(N)$	e7ad79, 13 lines
--	------------------

<pre> array<vi, 2> manacher(const string& s) { int n = sz(s); array<vi,2> p = {vi(n+1), vi(n)}; rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) { int t = r-i+!z; if (i<r) p[z][i] = min(t, p[z][l+t]); int L = i-p[z][i], R = i+p[z][i]-!z; while (L>=1 && R+1<n && s[L-1] == s[R+1]) p[z][i]++, L--, R++; if (R>r) l=L, r=R; } return p; } </pre>	
---	--

MinRotation.h Description: Finds the lexicographically smallest rotation of a string. Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end()); Time: $\mathcal{O}(N)$	d07a42, 8 lines
---	-----------------

<pre> int minRotation(string s) { int a=0, N=sz(s); s += s; rep(b,0,N) rep(k,0,N) { if (a+k == b s[a+k] < s[b+k]) {b += max(0, k-1); break;} if (s[a+k] > s[b+k]) { a = b; break; } } return a; } </pre>	
---	--

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any zero bytes. If you don't need `get_lcp(a,b)` then comment out RMQ lines.

Time: $\mathcal{O}(n \log n)$

```
struct SuffixArray {
    vi sa, lcp, rank;
    int N;
    RMQ<int> *rmq;
    SuffixArray(string& s, int lim=256) : N(sz(s)) {
        int n = sz(s) + 1, k = 0, a, b;
        vi x(all(s)+1), y(n), ws(max(n, lim));
        rank.resize(n);
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        rep(i,1,n) rank[sa[i]] = i;
        for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
            for (k && k--, j = sa[rank[i] - 1];
                s[i + k] == s[j + k]; k++);

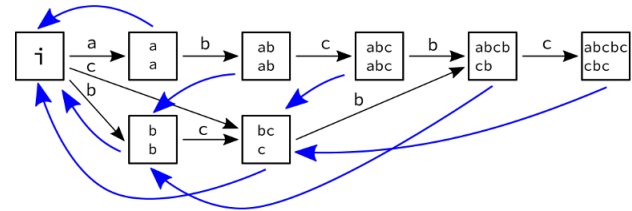
        rmq = new RMQ(lcp);
    }
    int get_lcp(int a,int b) {//lcp of suffixes starting at a,b
        if (max(a,b) >= N) return 0;
        if (a == b) return N-a;
        int t0 = rank[a] + 1, t1 = rank[b] + 1;
        if (t0 > t1) swap(t0,t1);
        return rmq->query(t0,t1);
    }
};
```

RunEnumerate.h

Description: Find all (i,p) such that `s.substr(i,p) == s.substr(i+p,p)`. No two intervals with the same period intersect or touch. Also look at comments below.

Time: $\mathcal{O}(N \log N)$

```
"SuffixArray.h"
vector<array<int,3>> solve(string s) {
    int N = sz(s); SuffixArray A(s);
    reverse(all(s));
    SuffixArray B(s);
    vector<array<int,3>> runs;
    for (int p = 1; 2*p <= N; ++p) { // do in O(N/p) for period p
        for (int i = 0, lst = -1; i+p <= N; i += p) {
            int l = i-B.get_lcp(N-i-p,N-i), r = i-p+A.get_lcp(i,i+p);
            if (l > r || l == lst) continue;
            runs.pb({lst = l,r,p}); // for each i in [l,r],
        } // s.substr(i,p) == s.substr(i+p,p)
    }
    return runs;
}
```



SuffixAutomaton.h

Description: Suffix automaton. Constructs a DAG efficiently maintaining equivalence classes of string occurrences. LOOK AT THE PICTURE!!! Each distinct string is some path through the automaton. Each occurrence of string `w` is a path from its node to some terminal node. At most $2N$ states and $3N$ edges in the whole automaton. Many things done by DP, add calculations in `init()`

Time: $\mathcal{O}(n\alpha)$ or $\mathcal{O}(n \log \alpha)$ If you need suffix tree, use suffix links in SA for reversed string.

```
<bits/stdc++.h>
using namespace std;
struct state {
    int len, link;
    map<char, int> next;
    state() : len(0), link(-1) {}
};
struct suffix_automaton {
    string input;
    vector <state> st;
    int last, size;
    vi top; vector<ll> cnt; vector<bool> odw;
    suffix_automaton(const string &s) : input(s), last(0), size
        (1) {
        st.push_back(state());
        trav(c, s) add_letter(c);
        init();
    }
    void dfs(int x) {
        odw[x] = 1;
        for (auto [lett, node] : st[x].next)
            if (!odw[node]) dfs(node);
        top.push_back(x);
    }
    void init() {
        int p = last;
        cnt.resize(size, 0); odw.resize(size, 0);
        while (p > 0) cnt[p]++, p = st[p].link;
        dfs(0);
        reverse(all(top)); assert(top[0] == 0);
        for (int i = sz(top)-1; i>0; --i) {
            for (auto [lett, node] : st[top[i]].next) {
                cnt[top[i]] += cnt[node]; //dp calculations here
            }
        }
    }
    void add_letter(char c) {
        st.push_back(state());
        int cur = size++;
        st[cur].len = st[last].len + 1;
        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        }
    }
};
```

```
} else {
    int q = st[p].next[c];
    if (st[p].len + 1 == st[q].len) {
        st[cur].link = q;
    } else {
        st.push_back(state());
        int clone = size++;
        st[clone].len = st[p].len + 1;
        st[clone].next = st[q].next;
        st[clone].link = st[q].link;
        while (p != -1 && st[p].next[c] == q) {
            st[p].next[c] = clone;
            p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}
int search(const string &s) {
    int q = 0;
    trav(c, s) {
        if (st[q].next.find(c) == st[q].next.end()) return 0;
        q = st[q].next[c];
    }
    return q;
}
ll count_occs(string &s) { return cnt[search(s)]; }
string lcs(const string &T) {
    int v = 0, l = 0, best = 0, bestpos = 0;
    rep(i, 0, sz(T)) {
        while (v && !st[v].next.count(T[i])) {
            v = st[v].link;
            l = st[v].len;
        }
        if (st[v].next.count(T[i])) {
            v = st[v].next[T[i]];
            l++;
        }
        if (l > best) {
            best = l;
            bestpos = i;
        }
    }
    return T.substr(bestpos - best + 1, best);
}
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

Time: $\mathcal{O}(n)$

```
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)
```



```

struct HashInterval {
    vector<H> ha, pw;
    HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
        pw[0] = 1;
        rep(i,0,sz(str))
            ha[i+1] = ha[i] * C + str[i],
            pw[i+1] = pw[i] * C;
    }
    H hashInterval(int a, int b) { // hash [a, b]
        return ha[b] - ha[a] * pw[b - a];
    }
};

vector<H> getHashes(string& str, int length) {
    if (sz(str) < length) return {};
    H h = 0, pw = 1;
    rep(i,0,length)
        h = h * C + str[i], pw = pw * C;
    vector<H> ret = {h};
    rep(i,length,sz(str)) {
        ret.push_back(h = h * C + str[i] - pw * str[i-length]);
    }
    return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}

```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. **Time:** construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

f35677, 66 lines

```

struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int& m = N[n].next[c - first];
            if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if (N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1) {
        rep(i,0,sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
        N.emplace_back(0);

        queue<int> q;
        for (q.push(0); !q.empty(); q.pop()) {
            int n = q.front(), prev = N[n].back;

```

```

        rep(i,0,alpha) {
            int &ed = N[n].next[i], y = N[prev].next[i];
            if (ed == -1) ed = y;
            else {
                N[ed].back = y;
                (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                    = N[y].end;
                N[ed].nmatches += N[y].nmatches;
                q.push(ed);
            }
        }
    }
}

vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}

vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};

```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). **Time:** $\mathcal{O}(\log N)$

edce47, 23 lines

```

set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L,R});
}

```

```

void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
}

```

```

    if (R != r2) is.emplace(R, r2);
}

```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty). **Time:** $\mathcal{O}(N \log N)$

9e9d8d, 19 lines

```

template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}

```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...}); **Time:** $\mathcal{O}(k \log \frac{n}{k})$

753a4c, 19 lines

```

template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to; p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}

template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}

```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a,b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B). **Usage:** int ind = ternSearch(0,n-1,[&](int i){return a[i]}); **Time:** $\mathcal{O}(\log(b-a))$

9155b4, 11 lines

```

template<class F>
int ternSearch(int a, int b, F f) {

```

```
assert(a <= b);
while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
}
rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
return a;
}
```

LIS.h
Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i,0,sz(S)) {
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

10.3 Dynamic programming

KnuthDP.h
Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h
Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$.
Time: $\mathcal{O}((N + (hi - lo)) \log N)$

```
struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });`
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K))`
if `(i & 1 << b) D[i] += D[i^(1 << b)];`
computes all sums of subsets.

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).
- `#pragma GCC target ("avx2,popcnt,lzcnt,abm,bmi,bmi2,fma,tune=native")` `dopychanie kolanem + 03 i unroll`

FastMod.h
Description: Compute $a \% b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h
Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as `cin/scanf`.

```
int readInt() {
    int a, c;
    while ((a = getchar_unlocked()) < 40);
    if (a == '-') return -readInt();
    while ((c = getchar_unlocked()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}
```

PQComp.h
auto `cmp = [](const pll &l, const pll &r) { return l > r; };`
`priority_queue<pll, vt<pll>, decltype(cmp)> q(cmp);`

CustomHash.h
`namespace std {`
 `template<`
 `struct hash<tuple<int, int, int>> {`
 `size_t operator() (const tuple<int, int, int> &t) const`
 `{`
 `return (1LL*get<0>(t)*1696969*1696969) ^ (1LL*get`
 `<1>(t)*1696969) ^ get<2>(t);`
 `}`
 `};`
 `}`
`}`

RandomTree.h
Usage: `auto edges = random_tree(n);` // `n ->` number of vertices
Time: $\mathcal{O}(n)$
`vt<pii> random_tree(int n) {`
 `vt<pii> edges(n-1);`
 `vt<int> id(n);`
 `iota(all(id), 1);`
 `shuffle(all(id), rng);`
 `for(int i=0; i<n-1; ++i)`
 `edges[i] = {id[i], id[rand(i+1,n-1)]};`
 `return edges;`
`}`